POT JS
FUNCTIONS

1.0.10

POT JS function descriptions with examples.
For the introduction and a tutorial, see the manual.

ASYNCHRONOUS FUNCTIONS

The core functions – `put()`, `get()`, `delete()`, `save()`, `load()` – return a promise. If an error is encountered, the promise is rejected, and an `Error` object is thrown. All functions whose names do *not* end in `Sync` are asynchronous. Their use is preferred.

Asynchronous functions have a timeout and a `cancel()` method. The timeout is the last parameter, the cancel method belongs to the promise object that the functions return. See pg. 4.

SYNCHRONOUS FUNCTIONS

There are synchronous variants for most functions, that add `Sync` to the function name: `putSync()`, `getSync()` etc. They can make sense for fast prototyping and can be used *in* synchronous functions.

But synchronous functions do not work in the browser when connecting to a network (as opposed to working with an in-memory POT). The connection is determined by using `pot.Kvs()` with or without its optional parameters. `pot.Kvs()` and `pot.Kvs(null, null)` effect that in-memory storage is used.

Synchronous calls never throw errors, they are returning an `Error` object instead in case of a failure.

TYPE-AWARE FUNCTIONS

`get()` returns the type that was stored using `put()`. The type information is persisted with the value. A value can have the type `boolean`, `number`, `string`, `Uint8Array`, or `null`.

RAW FUNCTIONS

The standard functions – `put()` and `get()` – operate with type information that is being written and read as first byte of the raw byte value that is actually being stored.

But functions with `Raw` in their name store and return the bytes as they are. The type information byte, if present, is misinterpreted as part of the value proper. These functions exist to design structures or attach meta data to values that is more complex than just the type. Certain functions – `getBoolean()`, `getString()`, `getNumber()` – read a raw byte value assuming that it will be a boolean, string, or number respectively. They assume that the raw data does not include a type code.

Standard and raw calls should not be mixed to avoid data corruption. A project should decide for one or the other to err on the safe side. Standard typed-coded are `put()`, `putSync()`, `get()`, `getSync()`. Raw are `putRaw()`, `putRawSync()`, `getRaw()`, `getRawSync()`, `getBoolean()`, `getBooleanSync()`, `getString()`, `getStringSync()`, `getNumber()`, `getNumberSync()`.

CANCELING

A promise returned by `put*()`, `get*()`, `delete()`, `save()` or `load()` is cancelable. These promise objects have `cancel()` methods. But the way promises work, there are two caveats:

Upon canceling, the promise rejects and if a **`.catch()`** method is chained, it is executed.

```
promise = kvs.put("fox", "The quick brown fox jumps!")
promise.catch(...)
...
promise.cancel()
```

1. But note the second line in the above example. Do **not** chain a `.then()` or a `.catch()` directly onto the `put()` call as it will return a different promise (to the variable `promise`): namely, the one that `.catch()` wraps around the promise returned by `put()`, which does not have the `cancel()` method. Instead, take the reference to the return value of `put()` first and then chain the `.catch()`.

For clarity, this does **not** work, the `.catch()` must not be chained in one go:

```
promise = kvs.put("fox", "The quick brown fox jumps!").catch(...)
...
promise.cancel() // TypeError: promise.cancel is not a function
```

The object in `promise` does not have the method `cancel()` because it is the promise that `.catch()` wrapped around the original promise returned by `put()`, which latter has the `cancel()` method.

2. When using **`try/catch`** instead, then unless `await` is used on the promise in the `try` block, the `catch` block does not catch the rejection. It does not matter from where the `cancel()` is called, within or from out of the `try` block. This `cancel()` is **not** caught:

```
try {
      promise = kvs.put("fox", "jumps!")
      promise.cancel()
  } catch(e) {
      // the cancel() does not trigger this catch block
  }
```

Such uncaught rejection can be caught by setting a global `unhandledRejection` handler, which is better than having the program crash from the uncaught exception:

```
process.on("unhandledRejection", (event) => {  ...  }).
```

The following `cancel()` **is** caught, because `await` is used:

```
try {
      promise = kvs.put("fox", "jumps!")
      setTimeout(promise.cancel(), 1)
      await promise
  } catch(e) {
      // the cancel() does trigger this catch block
  }
```

When trying this out, if the `put()` succeeds immediately, the `cancel()` might have no effect and not reject because the promise is already fulfilled.

CREATE AND SAVE KVS

NEW KVS

new pot.Kvs([<bee url>, <batch id>]) ⟶ KVS or error

<bee url> : http address of the API of a Bee Swarm client.
<batch id> : 64-digit hex string of a Swarm postage batch.

Create a new KVS object.

The parameters are optional. Leaving them out creates a key-value store in-memory. Giving both, connects to a Swarm Bee node to store the KVS there (with `save()`). `new pot.Kvs(null, null)` also selects in-memory storage.

In-memory:

```
(async () => {

      await pot.ready()
      kvs = new pot.Kvs()
      ...
})()
```

Connecting to a Swarm network. In the example, a local network:

```
(async () => {

      await pot.ready()
      const bee_url = "http://localhost:1633"
      const batch_id =
      "6792a183adafdac3a0a1a1e65b435406aff8f4343f980b16cabafd4a8525c0aa"
      kvs = new pot.Kvs(bee_url, batch_id)
      ...
})()
```

ALTERNATE NEW FUNCTIONS

pot.new([<bee url>, <batch id>]) ⟶ promise of KVS

pot.newSync([<bee url>, <batch id>]) ⟶ KVS or Error

<bee url> : http address of the API of a Bee Swarm client.
<batch id> : 64-digit hex string of a Swarm postage batch.

These functions are used to implement the `pot.Kvs()` constructor.

The parameters are optional. Leaving them out creates a key-value store in-memory. Giving both, connects to a Swarm Bee node to store the KVS there (with `save()`). `new(null, null)` also selects in-memory storage.

Although a promise is returned, `new()` does not accept a timeout as it is independent of external events and resolves immediately.

In-memory:

```
(async () => {

      await pot.ready()
      kvs = await pot.new()
      ...
})()
```

Connecting to a Swarm network. In the example, a local network:

```
(async () => {

      await pot.ready()
      const bee_url = "http://localhost:1633"
      const batch_id =
      "6792a183adafdac3a0a1a1e65b435406aff8f4343f980b16cabafd4a8525c0aa"
      kvs = await pot.new(bee_url, batch_id)
      ...
})()
```

Synchronous use:

```
globalThis.onPotInitialized = () => {

      kvs = pot.newSync(bee_url, batch_id)
      ...
}
```

LOAD

Create a new Swarm KVS from a save reference.

`pot.load(<reference>[, <bee url>, <batch id>[, <timeout>]])`

`⟶ cancelable promise of KVS`

`pot.loadSync(<reference>[, <bee url>, <batch id>[, <timeout>]])`

`⟶ KVS or Error`

`<reference>` : 32-digit hex string obtained from save().
`<bee url>` : http address of the API of a Bee Swarm client.
`<batch id>` : 64-digit hex string of a Swarm postage batch.
`<timeout>` : milliseconds.

The bee url and batch id parameters have to match (obviously) where the KVS was stored-to that is to be retrieved. That is, what was set in the constructor `pot.Kvs()` call originally. There is no direct way to migrate KVSs between different storages.

The promise returned by `load()` is cancelable, it has a `cancel()` method. When it is called and the promise had not resolved, a rejection is thrown. Learn how to use it right on pg. 4.

The optional timeout is in milliseconds. The loading will be canceled if loading is not complete before that timeout. For the async call, the promise will reject, throwing an error, the sync call will return an error. `bee_url` and `batch_id` can be given as `null` to give the timeout as fourth argument. Like with all functions that have a timeout, if no timeout is given, `load()` for its part never times out but other underlying mechanisms may at some point still timeout, throwing an error or dying silently.

```
ref = await kvs.save()
kvs2 = await pot.load(ref)
kvs3 = await pot.load(ref, null, null, 3000) // 3 sec timeout
      .catch(...)

kvs = new pot.Kvs()
kvs.putSync(key1, val1)
ref = kvs.saveSync()
kvs2 = pot.loadSync(ref)
```

SAVE

`kvs.save([<timeout>]) ⟶ cancelable promise of reference`

`kvs.saveSync([<timeout>]) ⟶ reference or Error`

`<timeout>` : milliseconds.
`<reference>` : 32-digit hex string.

Actually storing a KVS to the network. Put actions always operate in-memory first, until the entire map is saved to the network. Save does not accept a `bee_url` or `batch_id` argument. It uses what was determined when the KVS was created with `new pot.Kvs()` or `load()`. This function is not meaningful for in-memory settings but it exists also for KVS created in-memory to ease development and testing.

For a discussion of the save reference, see the manual, chapter Instructions.

The promise returned by `save()` is cancelable, it has a `cancel()` method. When it is called and the promise had not resolved, a rejection is thrown. Learn how to use it right on pg. 4.

The optional timeout is in milliseconds. The saving will be canceled if it is not complete before that time. For the async call, the promise will reject, throwing an error, the sync call will return an error.

```
await kvs.put(key1, val1)
ref = await kvs.save(100)     // one tenth of a second as timeout
kvs.putSync(key2, val2)
ref2 = kvs.saveSync()
```

STORING, RETRIEVING, AND DELETING KEY-VALUE PAIRS

The asynchronous, type-aware methods – `get()` and `put()` – are the preferred functions for production use, unless you are experimenting or really need the other ones.

PUT

Store a value, asynchronously, and type-aware.

After storing with `put()`, `get()` automatically converts the stored value back to the right type. (`getRaw()` would return all bytes – including the first byte that stands for the type – as `Uint8Array`.)

`kvs.put(<key>, <value>[, <timeout>]) ⟶ cancellable promise of null`

`kvs.putSync(<key>, <value>[, <timeout>]) ⟶ null or Error`

`<key>` : string, number, or Uint8Array. 32 bytes max.
`<value>` : string, number, Uint8Array, Boolean, or null.
`<timeout>` : milliseconds.

Keys are zero-padded to 32 bytes. Values can be up to 100,000 bytes or more. See the manual, chapter Instructions, for discussions of key ambiguity and value size.

The promise returned by `put()` is cancelable, it has a `cancel()` method. When it is called and the promise had not resolved, a rejection is thrown. Learn how to use it right on pg. 4.

The optional timeout is in milliseconds. The `put` will be canceled if it is not complete before that time. For the async call, the promise will reject, throwing an error, the sync call will return an error. If no timeout is given, `put()` never times out.

```
kvs = new pot.Kvs()
try {
    await kvs.put("fox", "The quick brown fox jumps!")
} catch(e) {
    ...
}
```

GET

`kvs.get (<key>[, <timeout>]) ⟶ cancelable promise of value`

`kvs.getSync(<key>[, <timeout>]) ⟶ value or Error`

`<key>` : string, number, or Uint8Array. 32 bytes max.
`<value>` : string, number, Uint8Array, Boolean, or null.
`<timeout>` : milliseconds.

Retrieve a value from the KVS, asynchronously and type-aware. (The first of the stored bytes is interpreted as type information, as written with `put()`, but not `putRaw()`).

The promise returned by `get()` is cancelable, it has a `cancel()` method. When it is called and the promise had not resolved, a rejection is thrown. Learn how to use it right on pg. 4.

The optional timeout is in milliseconds. The `get` will be canceled if the `get` is not complete before that timeout. For the async call, the promise will reject, throwing an error, the sync call will return an error. If no timeout is given, `get()` never times out.

```
try {
	value = await kvs.get(key)

} catch(e) {
	...

result = kvs.getSync(key)
if(result instanceof Error) {
	...
```

DELETE

kvs.delete (<key>[, <timeout>]) ⟶ cancelable promise of null

kvs.deleteSync(<key>[, <timeout>]) ⟶ null or Error

<key> : string, number, or Uint8Array. 32 bytes max.
<timeout> : milliseconds.

Drop a key-value pair from the key-value store.

The promise returned by `delete()` is cancelable, it has a `cancel()` method. When it is called and the promise had not resolved, a rejection is thrown. Learn how to use it right on pg. 4.

The optional timeout is in milliseconds. The deletion will be canceled if it is not complete before that timeout. For the async call, the promise will reject, throwing an error, the sync call will return an error. If no timeout is given, `delete()` never times out.

```
try {
	await kvs.delete(key)
} catch(e) {

result = kvs.deleteSync(key)
if(result instanceof Error) {
	...
```

ON INITIALIZATION

pot.ready() ⟶ promise of null

Use `pot.ready()` with `await` to wait until the pot object is initialized. Per design, the WASM executable has to be started and complete its main routine. This is triggered by `pot-node.js` or `pot-web.js`. Once this is done, `pot.ready()` resolves and the `pot.*` methods can be used from that point on.

```
await pot.ready()
kvs = new pot.Kvs()
...
```

globalThis.onPotInitialized() ⟶ none

This function is called from the WASM executable when the initialization of the `pot` object is done. It can be used as a synchronous substitute for `pot.ready()`. But it can also be defined asynchronously to allow for `await` in its body. If you don't define this function there is no harm.

```
globalThis.onPotInitialized = () => {
    kvs = new pot.Kvs()
    kvs.putSync("hello", "POT!")
    ...
}
```

RAW BYTE ACCESS

To store `Uint8Arrays` directly – e.g., to attach custom meta data beyond type information, or to inspect the byte level beneath the type-encoding as-is – values can be stored and retrieved as 'raw' bytes without the leading, type encoding byte that `put()`, `get()`, `putSync()`, and `getSync()` use to identify types. The raw mode is thus not compatible with the standard, type-aware access and raw values should not be retrieved with `get()` or `getSync()`.

Instead, these values can be retrieved raw, by `getRaw()` and then casted, or by having them converted immediately to a desired type with `getBoolean()`, `getNumber()`, and `getString()`.

Retrieving values with `get()` or `getSync()` that were stored with `putRaw()` leads to errors when the first byte gets interpreted as type information and then discarded. The same misinterpretation arises vice-versa, when a value stored with `put()` or `putSync()` is retrieved with `getBoolean()`, `getString()` or `getNumber()`. In these cases, the type code byte would be mixed into the value and a wrong value be returned. You might decide to use only one set of functions or the other in any given project to safely duck a nasty class of errors.

STORE RAW BYTES

kvs.putRaw(<key>, <value> : Uint8Array[, <timeout>])
⟶ cancelable promise of null

kvs.putRawSync(<key>, <value> : Uint8Array[, <timeout>])
⟶ null or Error

<key> : string, number, or Uint8Array. 32 bytes max.
<value> : Uint8Array.
<timeout> : milliseconds.

Store a raw `Uint8Array byte` array, without type information.

The promise returned by `putRaw()` is cancelable, it has a `cancel()` method. When it is called and the promise had not resolved, a rejection is thrown. Learn how to use it right on pg. 4.

The optional timeout is in milliseconds. The put will be canceled if it is not complete before that timeout. For the async call, the promise will reject, throwing an error, the sync call will return an error. If no timeout is given, `putRaw()` never times out.

RETRIEVE FROM RAW BYTES

`kvs.getRaw(<key>[, <timeout>]) ⟶ cancelable promise of Uint8Array`

`kvs.getRawSync(<key>[, <timeout>]) ⟶ Uint8Array or Error`

```
<key>          : string, number, or Uint8Array. 32 bytes max.
<value>        : Uint8Array.
<timeout>      : milliseconds.
```

Retrieve a value as raw `Uint8Bytes` array. Not expecting a type-byte. You can retrieve any value like this, but values stored with `put()` will have as first byte the type-information and the value itself starts with the second byte.

The promise returned by `getRaw()` is cancelable, it has a `cancel()` method. When it is called and the promise had not resolved, a rejection is thrown. Learn how to use it right on pg. 4.

The optional timeout is in milliseconds. The operation will be canceled if it is not complete before the timeout. For the async call, the promise will reject, throwing an error, the sync call will return an error. If no timeout is given, the method will not timeout.

`kvs.getBoolean(<key>[, <timeout>]) ⟶ cancelable promise of boolean`

`kvs.getBooleanSync(<key>[, <timeout>]) ⟶ boolean or Error`

```
<key>          : string, number, or Uint8Array. 32 bytes max.
<value>        : boolean.
<timeout>      : milliseconds.
```

Retrieve a value assumed to be stored raw, as a Boolean value. If the first byte is 0, `false` is returned, in all other cases, `true`.

The promise returned by `getBoolean()` is cancelable, it has a `cancel()` method. When it is called and the promise had not resolved, a rejection is thrown. Learn how to use it right on pg. 4.

The optional timeout is in milliseconds. The operation will be canceled if it is not complete before the timeout. For the async call, the promise will reject, throwing an error, the sync call will return an error. If no timeout is given, the method will not timeout.

Use `get()` to read a Boolean back that you stored with `put()`, it will automatically come back as `boolean`. Use `getBolean()` to read a Boolean back that was stored with `putRaw()`.

kvs.getNumber(<key>[, <timeout>]) ⟶ cancelable promise of number

kvs.getNumberSync(<key>[, <timeout>]) ⟶ number or Error

<key> : string, number, or Uint8Array. 32 bytes max.
<value> : number.
<timeout> : milliseconds.

Retrieve the raw bytes as floating-point number. The value has to be 8 bytes long.

The promise returned by `getNumber()` is cancelable, it has a `cancel()` method. When it is called and the promise had not resolved, a rejection is thrown. Learn how to use it right on pg. 4.

The optional timeout is in milliseconds. The operation will be canceled if it is not complete before the timeout. For the async call, the promise will reject, throwing an error, the sync call will return an error. If no timeout is given, the method will not timeout.

Use `get()` to read a number back that you stored with `put()`, it will automatically come back as number. Use `getNumber()` to read a number back that was stored with `putRaw()`.

kvs.getString(<key>[, <timeout>]) ⟶ cancelable promise of string

kvs.getStringSync(<key>[, <timeout>]) ⟶ string or Error

<key> : string, number, or Uint8Array. 32 bytes max.
<value> : string.
<timeout> : milliseconds.

Retrieve raw bytes as string.

The promise returned by `getString()` is cancelable, it has a `cancel()` method. When it is called and the promise had not resolved, a rejection is thrown. Learn how to use it right on pg. 4.

The optional timeout is in milliseconds. The operation will be canceled if it is not complete before the timeout. For the async call, the promise will reject, throwing an error, the sync call will return an error. If no timeout is given, the method will not timeout.

Use `get()` to read a string back that you stored with `put()`, it will automatically come back as string. Use `getString()` to read a string back that was stored with `putRaw()`.

SUPPORT FUNCTIONS

HELLO

pot.hello()

Write `hello` to browser console or terminal to verify that the WASM executable is up and running.

LOG

pot.log(message : string[, <log level : int>[, <no abbreviation : Boolean>]])

Verbosity-sensitive logging. "`pot: `" is prepended for every message.

This functions also serves to verify the successful language-lands roundtrip. If the log message arrives in the console or terminal, the WASM connection from Javascript works.

```
pot.log("hello!")
```

`pot.log()` is subject to the system verbosity setting. The default log level that a call to this `pot.log()` function uses is `INFO`. A call to `pot.log()` will not show up if the verbosity setting of the system is set to a value lower than the call's `<log level>`. In other words, the lower the setting of <log level>, the more likely that the log message will actually show up.

```
pot.log("hello.", pot.ERROR)
```

This example means that this message will appear in the console or terminal unless the verbosity setting is `NONE`, or `CRITICAL`.

For the verbosity log levels see `setVerbosity()`, pg. 17.

VALUE SIZE LIMIT

pot.setValueSizeLimit(limit : int) ⟶ previous limit : int

pot.getValueSizeLimit() ⟶ limit : int

Set the limit beyond which a value is rejected with an error. This function sets an arbitrary value to protect an application. The limit is in bytes (not characters).

The factual value size limit depends on the underlying system (OS, hardware) setup. For network use with node.js it is beyond 100,000 byte; for in-browser use, with network, it is beyond 10,000,000.

STRING BYTE SIZE

pot.byteSize(some : string) ⟶ size : int

The key size limit is 32 byte, not 32 characters. UTF-8 character encoding can use more than one byte per character and it is doable but not obvious and strangely arcane how the byte size of a string is determined in Javascript. It is also more of a lucky coincidence that Javascript's byte count is even relevant, i.e. that Javascript and Go use the same character encoding, UTF-8. To reduce complexity and sources of error, inquiry can be made with POT JS directly as to what the byte size of a string would be according to POT JS.

```
byteLen = pot.byteSize("€3") // 4
```

Eventually there is no harm in simply trying a key and handle the exception in case it is too long.

TRUNCATE STRING FOR BYTE SIZE

pot.truncString(some : string, size : int) ⟶ shortened : string

For the reasons mentioned immediately above, a function is provided in `truncString()` to cut a string to a given size in bytes, and cut cleanly along character borders also of multi-byte characters without producing invalid character codes.

```
key = pot.truncString(candidate, 32)
```

MEMORY MANAGEMENT

GARBAGE COLLECTION

pot.gc() ⟶ null

This function triggers the Go garbage collector. Depending on the amount of data stored in memory, garbage collection can take some split seconds.

The garbage collection will automatically trigger at some point. Calling `gc()` can reduce the likeliness that the system freeze of garbage collection happens at a later, maybe less suitable moment. Especially when using the in-memory storage, some 100,000 key-value pairs can effect a notable system pause when garbage is collected.

The Javascript garbage collector cannot be called directly but Javascript should have a much smaller heap size that does not grow directly from POT JS key-value pairs being operated on or stored.

If optimization (see below) is set to `false`, the call to `gc()` does nothing.

STATE PRUNING

pot.prune() ⟶ null

A long-running, heavy duty server may benefit from occasional (24h) pruning of its internal KVS-state map. The background is that Go maps do not shed the internal 'buckets' of entries pairs that were added to a map structure, even if the entry is deleted. Pruning copies the old state map over to a new map to free the lost memory. If the state map is very large – if a lot of key-value stores are active or have been active – the function may take some milliseconds. It is not affected by the number of key-value pairs stored or handled.

If optimization (see below) is set to `false`, the call to `prune()` does nothing.

STATE PURGE

pot.purge() ⟶ null

The KVS state stored in-memory can be dropped for all KVS with this function. This leads to a loss of all stored key-value pairs, they are not backed up in any way. Relevantly this deletes the residue of key-value pairs that were part of a KVS but have now been deleted, or were part of a KVS that has gone out of scope. Any pair that was stored remains in-memory, emulating in this regard an immutable storage network.

This method exists for memory tests.

KVS RELEASE

kvs.release() ⟶ null

The internal KVS state (of a KVS connected with a network or in-memory) can be released with this function. This leads to an error, if the KVS is used after this point. The KVS is being treated as if it had become unreachable and is being garbage collected.

This method exists to expedite garbage collection and for memory tests. It is similar to a weak reference in that it may be useful if a KVS cannot be made unreachable but should still be collected.

HEAP SIZE INFORMATION

pot.getJSHeapSize() ⟶ number

Returns the current size of the Javascript heap. Uses `process.memoryUsage.heapUsed` or `performance.memory.usedJSHeapSize` as available.

pot.getGoHeapSize() ⟶ number

Returns the current size of the Go heap.

pot.profile() ⟶ string

Returns a string with Javascript and Go heap size and miniature bar charts, like:

Go heap 8944K IIIIIIIII JS heap 18647K ▮IIIIIIIII

pot.bar(n : int) ⟶ string

Returns a miniature bar chart as used in profile(), e.g. ▮IIIIIIIII for n = 18647000.

OPTIMIZATION SETTING

To potentially increase robustness, switching off optimization allows to switch off resource releases to possibly achieve more robust operation at the cost of minor resource leakage. For a discussion of the dimension of the leakage, see the manual. Basically, some channels and structures are not released per KVS. This means that leakage is negligible if you don't work with many KVS and non-existent if you never let KVS variable become unreachable before program exit at any rate.

This option exists in case resource releases are suspected to cause internal errors. This has not been observed.

The default setting is to release resources, the STANDARD level.

`0`	`NONE`	resources are not released, the system leaks slightly, might be more robust.
`1`	`STANDARD`	resources are released, this is the expectable and default behavior.

There are two ways to set optimization. It can be changed anytime with:

`pot.setOptimization(<0 or 1>) ⟶ previous optimization setting`

`pot.getOptimization() ⟶ optimization setting`

```
(async () => {

      await pot.ready()
      pot.setOptimization(pot.NONE) // or 0
      map = new pot.Kvs()

})()
```

`globalThis.potOptimization = <0 or 1>`

To set Optimization before the WASM executable is initialized, to suppress even the first two lines it would otherwise log, use `potOptimization`. This works only when it is set before there WASM executable is started. Setting it to a new value after that will not have an effect.

```
var go = new Go()

global.potOptimization = 0 // `pot.NONE` does not exist yet

WebAssembly.instantiate(
      fs.readFileSync("lib/pot.wasm"), go.importObject)
        .then((r) => { go.run(r.instance) })

onPotInitialized = async () => {

      ...
```

VERBOSITY

Verbosity controls the level of detail of the logs written to screen or browser console.

The default setting is `DEBUG`-level verbosity, which results into a high amount of log entries to track a lot of details. For production, `INFO` or `ERROR` will make sense.

There is no general need to allow even errors to make a log entry, as they are also thrown or returned as value by any function call that is affected. `NONE` can therefore be a valid choice.

Levels

Verbosity can be set to these levels:

`0`	`pot.NONE`	no logging, not even errors.
`1`	`pot.CRITICAL`	logs only errors that appear to arise from a POT JS malfunction.
`2`	`pot.ERROR`	programming and runtime errors are also logged.
`3`	`pot.INFO`	general runtime information is logged.
`4`	`pot.DEBUG`	specific data, like put and get keys and values are logged.
`5`	`pot.TRACE`	some execution paths or logged and hex values are not abbreviated.

There are four ways to set the level that are useful in different situations. The examples in `examples/` and the Examples chapter in the manual illustrate the uses.

In general, verbosity can be changed anytime with:

`pot.setVerbosity(level : int) ⟶ previous verbosity setting`

`pot.getVerbosity() ⟶ verbosity setting`

```
(async () => {
      await pot.ready()
      pot.setVerbosity(pot.INFO) // or 3
      map = new pot.Kvs()
})()
```

All of the following ways to set verbosity cater to the potential wish to suppress also the first couple of lines that POT JS logs during initialization of the WASM executable.

`globalThis.potVerbosity = level : int`

In the special case that `pot-node.js` or `pot-web.js` are not used, to set verbosity before the WASM executable is initialized, to suppress even the first lines it would otherwise log, use `globalThis.potVerbosity`. Of course, this works only when it is set before the WASM executable is started. Setting it to a new value after that will not have an effect.

```
var go = new Go()
globalThis.potVerbosity = 3 // can't be pot.INFO, it does not exist yet
WebAssembly.instantiate(
      fs.readFileSync("lib/pot.wasm"), go.importObject)
       .then((r) => { go.run(r.instance) })

onPotInitialized = async () => {
...
```

<script ... verbosity=level>

The verbosity level can be set in the `verbosity` attribute of the `script` tag that includes `pot-web.js`. Like with all ways to set verbosity before pot is initialized, it cannot use the pot.* constants but must be a literal.

```
<script src="lib/pot-web.js" wasm="lib/pot.wasm" verbosity="3"></script>
```

require(module)([wasm path,] level)

The verbosity level can also be set in a second parameter group to require, in a separate bracket, as first or second parameter. It cannot use the pot.* constants but must be a literal. In these examples it is in each case the 3:

```
require("pot-node")(3)

require("pot-node")("lib/pot.wasm", 3)
```

DEVELOPMENT

The following functions have no role in production use of POT JS. They are for testing POT JS.

GENERAL TESTING

These functions exist for black box testing. See `examples/suites.js` for examples of their use.

RANDOM KEY

`pot.randKey() ⟶ Uint8Array(32)`
Generate 32 random bytes for key testing.

RANDOM VALUE

`pot.randValue() ⟶ Uint8Array(79–101)`
79 to 101 random bytes for value testing (range taken from Go POT).

RANDOM BUFFER

`pot.randBuffer(n) ⟶ Uint8Array(n)`
Generate n random bytes for value testing.

TYPE-ENCODED BYTES

`pot.typeEncodedBytes(<value>) ⟶ Uint8Array`
Javascript type detection and value encoding with correct leading type byte.
`value types: string, number, Boolean, Uint8Array, null.`

TYPE-DECODED VALUE

`pot.typeDecodedValue(encoded bytes : Uint8Array) ⟶ value`
Javascript type detection by leading type byte and according conversion of raw KVS value.
`value types: string, number, Boolean, Uint8Array, null.`

SIMULATION TESTING

The following functions are effective only when the WASM executable has been compiled for the API-side simulation for extended testing. They exist but are all no-ops in the normal production build.

FAULTY PROMISES

`pot.hangingPromise() ⟶ promise of null`

Blocking promise (programmed in Go) with 1 second time out for exception testing.

This function exists but is a no-op in the normal production build.

`pot.panickingPromise() ⟶ rejected promise`

This promise (programmed in Go) immediately panics, i.e., simulates a Go exception.

This function exists but is a no-op in the normal production build.

SET FAULT

`pot.setFault(bool)`

Make the next function call to the mock storage function return an (JS) error to JS.

This function exists but is a no-op in the normal production build.

SET PANIC

`pot.setPanic(bool)`

Make the next function call to the mock storage function panic (on the Go side).

This function exists but is a no-op in the normal production build.

SET DELAY

`pot.setDelay(msec : int)`

Make the next call to a mock storage function stall for msec milliseconds.

This function exists but is a no-op in the normal production build.

SET HANG

`pot.setHang(bool)`

Make the next call to a mock storage function stall indefinitely.

This function exists but is a no-op in the normal production build.